

Модели стоимости доступа к многомерным данным в страничных индексных структурах

А. М. Бородин, С. В. Поршнева

Аннотация

Рассматривается обобщённое дерево поиска — структура, лежащая в основе индексирования многомерных данных в современных СУБД. Строятся аналитические модели стоимости поиска, построения и сопровождения индекса, выраженные через число обращений к страницам с разделением на произвольные и последовательные. Показано, что перекрытие ключей не накапливается с глубиной дерева, а выходит на характерное для стратегии разбиения установившееся значение, и потому входит в стоимость поиска постоянным множителем на каждом шаге спуска; ветвистость же узла влияет на стоимость поиска логарифмически, а на объём индекса и стоимость массовых операций — обратно пропорционально. Модели проверены измерениями: для двух стратегий разбиения при сортированном построении установившееся перекрытие составляет 4,0 и 1,3, что даёт двукратное различие стоимости точечного запроса, устойчивое при изменении объёма данных на порядок. Из моделей выделены три параметра, поддающиеся воздействию без изменения класса операторов: ветвистость, перекрытие и порядок обращения к страницам. Показано, что при росте размерности данных чувствительность стоимости поиска к качеству разбиения резко возрастает. Полученные соотношения дают критерий сопоставления методов построения индекса, отличный от общепринятого сравнения по времени построения.

Ключевые слова: многомерные данные, индексирование, обобщённое дерево поиска, R-дерево, модель стоимости, ветвистость, перекрытие ключей.

1 Введение

Многомерные данные перестали быть принадлежностью специализированных систем. Геометрические и географические объекты, диапазоны значений, полнотекстовые сигнатуры и векторные представления обрабатываются универсальными СУБД наравне со скалярными типами, и универсальным механизмом доступа к ним стало обобщённое дерево поиска [1]: ядро реализует страничную организацию, спуск, расщепление, конкурентный доступ и журналирование, а предметная часть выносится в класс операторов.

Теория качества таких деревьев развита: со времён R-дерева [2] известно, что стоимость поиска определяется перекрытием ключей узлов и покрытой ими площадью, и предложены критерии разбиения узла, эти величины улучшающие [3, 4]. Аналитическая модель предсказания стоимости запроса к R-дереву построена Теодоридисом и Селлисом [5]: число просмотренных узлов выражается через плотность данных и размеры ограничивающих прямоугольников, оцениваемые по выборке. Свод результатов этого направления дан в [6], обзор многомерных методов доступа — в [7].

Модель [5] была основанием и для более ранних работ авторов, где оценивалась эффективность агрегирующих запросов к аналитическим данным, отображённым в пространственные [8, 9, 10]; сопоставление с реальными измерениями на средствах бизнес-аналитики приведено в [11]. Настоящая работа продолжает эту линию, но меняет предмет: там оценивалась стоимость запроса к дереву заданной формы, здесь — зависимость этой формы от решений, принимаемых реализацией.

Общее свойство перечисленных моделей — они выражены в терминах свойств разбиения пространства (плотность, площадь, перекрытие) и не связаны с параметрами страничной организации узла, тогда как в реализации управлять можно именно последними. Разработчик СУБД не может изменить перекрытие непосредственно: он может изменить представление ключа, способ выбора точек разбиения, порядок обхода страниц при построении и сопровождении. Какое из этих действий и на сколько изменит наблюдаемую стоимость запроса — вопрос, на который существующие модели не отвечают.

Практическое следствие этого разрыва состоит в том, что методы построения индекса сравниваются по времени построения. Такое сравнение систематически вводит в заблуждение: способ построения влияет на форму дерева, то есть на стоимость всех последующих запросов, и выигрыш во времени построения может быть с избытком оплачен.

Цель работы — построить модели стоимости построения, поиска и сопровождения обобщённого дерева поиска, выраженные через параметры, которыми располагает реализация, и проверить их измерением на промышленной СУБД.

2 Постановка и обозначения

Рассматривается сбалансированное дерево, узел которого занимает страницу. Элемент внутреннего узла — пара «ключ, нисходящая ссылка»; ключ покрывает ключи всех элементов поддеревья. Поиск спускается во все поддеревья, ключи которых могут содержать ответ, поэтому путей спуска может быть несколько, и стоимость поиска определяется их числом, а не высотой дерева.

Класс операторов предоставляет операции `consistent` (может ли поддерево содержать ответ), `union` (ключ, покрывающий набор ключей), `penalty` (штраф за расширение ключа) и `picksplit` (разбиение переполненного узла). Порядка на множестве ключей интерфейс не содержит.

3 Ветвистость узла

Ветвистость определяется размером страницы и размером хранимого элемента:

$$f = \left\lfloor \frac{B - h}{k + p + \sigma} \right\rfloor, \quad H = \left\lceil \log_f \frac{N}{f} \right\rceil + 1, \quad L = \left\lceil \frac{N}{f\varphi} \right\rceil. \quad (1)$$

Слагаемое σ в теоретических работах обычно опускают. В реализации оно не равно нулю: ключ может храниться в преобразованном виде, вместе с ним могут храниться дополнительные атрибуты, а часть интерфейса класса операторов вынуждает хранить избыточные представления. Именно σ и k — те величины, на которые реализация воздействует непосредственно.

Из (1) видно, что ветвистость влияет на высоту логарифмически, а на число страниц — обратно пропорционально. Поэтому эффект от её роста проявляется не столько в сокра-

Таблица 1: Обозначения

N	число индексируемых записей
B	размер страницы
h	размер служебного заголовка страницы
k	средний размер ключа
p	размер ссылки
σ	служебные данные, сопровождающие элемент в узле
f	ветвистость узла
H	высота дерева
φ	коэффициент заполнения страницы
D	размерность данных
M	память, доступная сортировке
c_r, c_s	стоимость произвольного и последовательного чтения страницы
c_c	стоимость одного вычисления consistent

щении длины пути спуска, сколько в сокращении объёма индекса и стоимости операций, обходящих индекс целиком: построения, сборки мусора, верификации.

4 Стоимость поиска и перекрытие ключей

Определим перекрытие уровня l как математическое ожидание числа узлов этого уровня, ключи которых удовлетворяют условию поиска:

$$\omega_l = \mathbb{E}|\{u \in U_l : \text{consistent}(\text{key}(u), Q)\}|. \quad (2)$$

Для дерева без перекрытия $\omega_l = 1$ на всех уровнях. Стоимость точечного запроса составляет

$$P_{\text{point}} = \sum_{l=0}^{H-1} \omega_l c_r. \quad (3)$$

Узел посещается, только если посещён его родитель, что подсказывает вывод о накоплении перекрытия вниз по дереву. *Этот вывод неверен.* Измерение (§8) показывает, что ω_l выходит на характерное для способа построения значение ω_∞ уже на первом-втором уровне под корнем и далее с глубиной не растёт; верхние уровни составляют исключение лишь потому, что там ω_l ограничено числом узлов уровня. Следовательно

$$P_{\text{point}} \approx \omega_\infty H c_r, \quad (4)$$

и качество разбиения входит в стоимость поиска постоянным множителем на каждом шаге спуска, а не накапливающимся по уровням.

Различие между (3) и (4) существенно для практики. Мультипликативное накопление означало бы, что ухудшение разбиения тем опаснее, чем больше данных; постоянный множитель означает, что относительный проигрыш от плохого разбиения не зависит от объёма. Второе и наблюдается.

4.1 Зависимость от размерности

Предсказание (6) проверяется на классе операторов `cube` — единственном в поставке, работающем с данными переменной размерности. Сортированное построение для него недоступно, поэтому дерево строится вставкой. 500 000 точек, равномерно распределённых в единичном D -мерном кубе, 2000 зондов; оба множества порождены с фиксированным зерном. Запрос точечный: отыскиваются элементы, содержащие заданную точку. Ответ почти всегда пуст, но обход посещает ровно те узлы, чей ключ содержит точку, то есть $\sum_l \omega_l$ — это прямое измерение величины из модели, а не её следствия.

Таблица 2: Число обращений к страницам при точечном запросе в зависимости от размерности

D	f	H	страниц	обращений	$\bar{\omega}$
2	54,8	4	4 519	6,59	1,65
3	38,8	4	5 259	13,49	3,37
4	32,4	4	6 252	39,89	9,97
6	17,7	5	7 999	59,73	11,95
8	11,6	5	9 863	145,78	29,16
12	5,9	7	13 987	174,94	24,99
16	4,0	8	18 623	266,26	33,28
24	2,3	14	29 456	570,70	40,76
32	2,0	16	38 517	785,31	49,08

Число обращений выросло в 119 раз, но приписывать этот рост целиком формуле (6) нельзя: за ним стоят две различные причины, и их следует разделить.

Первая — размер ключа. Ключ `cube` размерности D занимает $16D$ байт, поэтому ветвистость падает с 54,8 до 2,0, а высота растёт с 4 до 16. При ветвистости 2 дерево вырождается в двоичное, и уровней приходится проходить больше просто по его устройству. Это следствие представления ключа, описываемое формулой (1), а не геометрии.

Вторая — собственно перекрытие. Нормировка на высоту даёт среднее число узлов, посещаемых на одном уровне: $\bar{\omega}$ растёт с 1,65 до 49,1, то есть в 30 раз. При идеальном разбиении ключи одного уровня не перекрывались бы и покрывали бы пространство ровно один раз, откуда $\bar{\omega} = 1$ при любом D ; при $D = 2$ измеренные 1,65 недалеко от этого, при $D = 32$ на каждом уровне просматривается полсотни узлов вместо одного. Это и есть содержание (6): показатель $1/D$ делает сторону ключа близкой к единице, отклонение разбиения от идеального перестаёт быть малым, и перекрытие растёт кратно.

Подтверждено, таким образом, направление и порядок величины, но не функциональная форма зависимости. Чтобы утверждать, что $\bar{\omega}$ растёт именно так, как следует из a_l^D , нужно измерять ω_l по уровням отдельно и знать фактические размеры ключей на каждом уровне; здесь измерено среднее по дереву.

4.2 Процессорная составляющая

Формула (3) учитывает только обращения к страницам. При данных, помещающихся в оперативной памяти, доминирует другая величина: чтобы отобрать поддеревья для спуска, ядро вычисляет `consistent` для каждого из f элементов узла. С учётом этого

$$T = \sum_{l=0}^{H-1} \omega_l (c_r + f c_c). \quad (5)$$

Из (5) следует ограничение, не видное в (3): рост ветвистости сокращает число обращений к страницам, но линейно увеличивает процессорную стоимость просмотра узла. Осмысленное увеличение размера страницы поэтому ограничено, и снятие этого ограничения требует внутривстраничной структуры, позволяющей отбирать кандидатов не за $O(f)$.

5 Влияние размерности

Пусть данные равномерно распределены в единичном D -мерном кубе, а ключ узла — ограничивающий параллелепипед со стороной a_l . Вероятность того, что случайный точечный запрос попадёт в ключ, равна a_l^D , откуда

$$\omega_l \approx |U_l| a_l^D, \quad a_l \approx \left(\frac{f^{l+1}}{N} \right)^{1/D}. \quad (6)$$

Показатель $1/D$ означает, что при больших D сторона ключа близка к единице уже на средних уровнях, и малое отклонение разбиения от идеального даёт кратный рост ω_l . Отсюда объяснение практического наблюдения: приёмы, безопасные для одномерного В-дерева, для многомерного дерева опасны, поскольку там качество разбиения входит в стоимость с показателем, растущим по размерности.

6 Стоимость построения

Построение последовательной вставкой спускается к листу для каждой записи:

$$P_{\text{ins}} = N (H + \gamma) c_r, \quad (7)$$

где γ — среднее число дополнительных обращений на расщепление и обновление ключей родителей. Обращения произвольные: порядок вставки не связан с физическим расположением страниц.

Если для класса операторов задан порядок, сохраняющий локальность, набор данных можно отсортировать и заполнять страницы подряд снизу вверх:

$$P_{\text{sort}} = \frac{2N}{B} \log_{M/B} \frac{N}{B} c_s + \frac{N}{f\varphi} c_s. \quad (8)$$

Источников выигрыша три: замена произвольного доступа последовательным, отказ от расщеплений и рост φ с характерных для расщепления 0,7 до заданного коэффициента заполнения.

6.1 Критерий сопоставления методов построения

Сравнение (7) и (8) неполно, поскольку метод построения влияет на ω_∞ , то есть на стоимость всех последующих запросов. Правильным основанием для сравнения является

$$P_{\text{build}} + n P_{\text{point}} = P_{\text{build}} + n \omega_\infty H c_r, \quad (9)$$

где n — ожидаемое число запросов за время жизни индекса. Из (9) следует, что метод построения, ускоряющий сборку в α раз ценой роста ω_∞ в β раз, оправдан лишь при

$$n < \frac{P_{\text{build}}(1 - 1/\alpha)}{(\beta - 1)\omega_\infty H c_r}, \quad (10)$$

то есть при достаточно коротком времени жизни индекса или достаточно редких запросах. Для индекса, строящегося однократно и обслуживающего миллионы запросов, правая часть (10) пренебрежимо мала, и любой рост ω_∞ неприемлем независимо от выигрыша в сборке.

7 Стоимость сопровождения

Сборка мусора обходит все страницы индекса. При обходе в логическом порядке последовательность номеров страниц произвольна, при обходе в физическом порядке — возрастающая:

$$P_{\text{vac}}^{\text{log}} = (L + I) c_r, \quad P_{\text{vac}}^{\text{phys}} = (L + I) c_s + \beta c_r, \quad (11)$$

где I — число внутренних страниц, β — число страниц, потребовавших повторной обработки из-за конкурентных вставок. Выигрыш пропорционален $(L + I)(c_r - c_s)$ и, следовательно, объёму индекса — то есть обратно пропорционален ветвистости, что связывает (11) с (1).

8 Экспериментальная проверка

Измерения выполнены на СУБД PostgreSQL, 16-ядерная машина, буферный кэш 8 Гиб. Первичная величина — число обращений к страницам индекса на запрос, полученное из плана выполнения. Она равна $\sum_l \omega_l$, не зависит от носителя и воспроизводится при фиксированном зерне генератора точно, в отличие от измерений времени.

8.1 Перекрытие не накапливается с глубиной

Таблица 3 приводит ω_l по уровням для десяти миллионов равномерно распределённых точек при трёх способах построения.

Таблица 3: Перекрытие по уровням, $N = 10^7$, 2000 зондов

Уровень	вставка		по заполнению		picksplit	
	узлов	ω_l	узлов	ω_l	узлов	ω_l
0	1	1,00	1	1,00	1	1,00
1	8	1,05	3	1,78	5	1,99
2	823	1,13	363	4,24	601	2,47
3	90 076	1,06	60 241	3,95	80 737	1,35

Каждый способ выходит на своё значение в пределах одного-двух уровней от корня и далее его удерживает: $\omega_\infty \approx 1,05$ для построения вставкой, $\approx 4,0$ для разбиения по

заполнению страницы, $\approx 1,3$ для разбиения функцией класса операторов. Это и есть эмпирическое содержание формулы (4).

Следствие проверяется независимо: при увеличении объёма данных на порядок высота дерева растёт, а отношение стоимостей между способами построения должно сохраниться. Измерено 12,67 против 6,86 обращений при 10^7 и 15,20 против 7,91 при $1,38 \cdot 10^8$ точках реальных данных, то есть отношение 1,85 и 1,92 соответственно.

8.2 Перекрытие, а не ветвистость

Стоимость запроса в измерениях монотонно убывает с ростом числа страниц индекса, что допускает истолкование через ветвистость: менее ветвистое дерево имеет меньше элементов в узле, и классическая оценка даёт оптимальную ветвистость около e . Для проверки коэффициент заполнения при разбиении по заполнению страницы понижался до сравнения числа страниц с тем, что даёт разбиение через `picksplit`.

Таблица 4: Проверка истолкования через ветвистость, $N = 10^7$

разбиение	φ	страниц	обращений
<code>picksplit</code>	по умолчанию	81 326	6,86
по заполнению	по умолчанию	60 608	12,67
по заполнению	65	84 036	13,64
по заполнению	50	109 892	14,49

При практически равном числе страниц (84 036 против 81 326) разбиение по заполнению требует вдвое больше обращений. Снижение коэффициента заполнения не улучшает стоимость запроса, а слегка ухудшает её. Истолкование через ветвистость не подтверждается: в стоимость входит качество разбиения, то есть ω_∞ , а не число страниц.

8.3 Процессорная составляющая

Профилирование серверного процесса на нагрузке из пространственных соединений, когда индекс и таблица целиком находятся в буферном кэше, даёт следующее распределение: перебор элементов страницы 22,4 %, вычисление предиката 14,1 %, поиск страницы в таблице буферов 12,8 %, закрепление страницы 5,7 %, вызов предиката через диспетчер функций 4,7 %. Работа с элементами страницы составляет в сумме 46,8 %.

Это подтверждает (5) и уточняет её: величина c_c складывается не столько из предметных вычислений, сколько из накладных расходов обхода. Прямая проверка следствия: реализация внутривстраничной структуры, позволяющей пропускать группы элементов, ускорила соединение по индексу в 1,41 раза при неизменном числе обращений к страницам, то есть выигрыш получен целиком за счёт слагаемого fc_c .

9 Границы применимости моделей

Модели проверены не полностью, и это следует назвать явно.

Зависимость от размерности (6) подтверждена качественно: измерено направление и порядок величины при D от 2 до 32, но не функциональная форма — поуровневые значения ω_l и фактические размеры ключей на уровнях не измерялись. Измерение выполне-

но для дерева, построенного вставкой: для класса операторов над данными переменной размерности сортированное построение недоступно.

Стоимость сопровождения (11) измерением не проверялась. Разделение произвольного и последовательного чтения требует носителя с известными характеристиками, тогда как измерения выполнялись на виртуальной машине, где отношение c_r/c_s определяется гипервизором, а не устройством.

Установившееся перекрытие измерено для одного класса операторов при равномерном и кластеризованном распределении и для одного набора реальных данных. Утверждение о том, что ω_l выходит на постоянное значение, опирается на деревья высотой три и четыре; для более высоких деревьев оно не проверялось.

Величина c_c в (5) измерена профилированием для одного класса операторов. Для классов с существенно более дорогим предикатом её доля должна быть выше, что согласуется с моделью, но не измерено.

10 Заключение

Построены модели стоимости построения, поиска и сопровождения обобщённого дерева поиска, выраженные через параметры страничной организации узла. Установлено, что перекрытие ключей не накапливается с глубиной дерева, а выходит на характерное для способа построения установившееся значение, и потому входит в стоимость поиска постоянным множителем на каждом шаге спуска.

Из моделей выделены три величины, поддающиеся воздействию без изменения класса операторов: ветвистость узла, установившееся перекрытие и порядок обращения к страницам. Показано, что при данных, помещающихся в оперативной памяти, ветвистость входит в стоимость дважды и с противоположными знаками: сокращая число обращений к страницам и увеличивая процессорную стоимость просмотра узла.

Предложен критерий сопоставления методов построения индекса, учитывающий влияние способа построения на стоимость последующих запросов, и получено условие (10), при котором ускорение построения ценой ухудшения дерева оправдано.

Все утверждения проверены измерением на промышленной СУБД; первичной величиной выбрано число обращений к страницам индекса как не зависящее от стека и воспроизводимое точно.

Список литературы

- [1] Hellerstein Joseph M, Naughton Jeffrey F, Pfeffer Avi. Generalized search trees for database systems. — University of Wisconsin-Madison, 1995.
- [2] Guttman Antonin. R-trees: a dynamic index structure for spatial searching // ACM SIGMOD Record / ACM. — 1984. — Vol. 14. — P. 47–57.
- [3] The R*-tree: an efficient and robust access method for points and rectangles / Beckmann Norbert, Kriegel Hans-Peter, Schneider Ralf, and Seeger Bernhard // ACM SIGMOD Record / ACM. — 1990. — Vol. 19. — P. 322–331.
- [4] Beckmann Norbert, Seeger Bernhard. A revised R*-tree in comparison with related index structures // Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data / ACM. — 2009. — P. 799–812.

- [5] Theodoridis Yannis, Sellis Timos. A model for the prediction of R-tree performance // Proceedings of the 15th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS) / ACM. — 1996. — P. 161–171.
- [6] R-trees: Theory and Applications / Manolopoulos Yannis, Nanopoulos Alexandros, Papadopoulos Apostolos N, and Theodoridis Yannis. Advanced Information and Knowledge Processing. — Springer, 2006. — P. 194.
- [7] Gaede Volker, Günther Oliver. Multidimensional access methods // ACM Computing Surveys. — 1998. — Vol. 30, no. 2. — P. 170–231.
- [8] Бородин А. М., Сидоров М. А., Поршнева С. В. Методы оценки эффективности R-tree индекса агрегирующих запросов в OLAP-системах // Научные труды международной научно-практической конференции «Связь-Пром 2009». — Екатеринбург. — 2009. — Vol. 2. — P. 59–66.
- [9] Бородин А. М., Поршнева С. В. Аналитические способы оценки эффективности применения пространственных индексов в OLAP-системах // Научно-технические ведомости СПбГПУ. Информатика. Телекоммуникации. Управление. — 2011. — no. 2 (120). — P. 93–100.
- [10] Бородин А. М. Алгоритмы быстрого доступа к многомерным данным в OLAP-системах. — LAP Lambert Academic Publishing, 2012. — P. 180.
- [11] Бородин А. М., Поршнева С. В. Сравнительный анализ возможностей и скорости обработки многомерных данных программными средствами бизнес-аналитики на основе индексирующих структур основной памяти // Научно-технические ведомости СПбГПУ. Информатика. Телекоммуникации. Управление. — 2010. — no. 1 (93). — P. 99–102.